



TEST CASE GENERATOR INTEGRATION WITH JIRA TO FETCH USER STORY

Week3-Day2 Assignment



JUNE 25, 2026
ANITHA PRAJISH

Contents

1. Objective	2
2. Task Overview	2
Frontend Changes.....	2
Backend Changes	2
AI-Assisted Development Workflow	2
3. Tools Used	3
4. Understanding Gained	3
AI-Assisted Development (Vibe Coding).....	3
Prompt Engineering.....	3
Existing Codebase Analysis	3
API Integration	3
LLM Application Development	3
5. Implementation Process Followed	4
6. Challenges Faced.....	4
7. Outcome	5
8. Key Learnings.....	7
9. Conclusion	8

1. Objective

The objective of this assignment was to understand how to implement Jira integration into an existing codebase while learning how to use AI-assisted development tools such as GitHub Copilot in Visual Studio Code to analyze code, suggest implementation steps, and generate code changes iteratively.

2. Task Overview

The assignment focused on enhancing an existing application by integrating Jira connectivity and using AI-assisted coding (Vibe Coding) to implement the changes.

The main requirements were:

Frontend Changes

- Add input fields for:
 - Base URL
 - Email ID
 - Jira API Key
- Add a **“Connect Jira”** button
- Establish Jira connection from the UI
- Add input fields for:
 - Work Type ID
- Add a **“Fetch Story”** button
- Populate the selected user story details automatically after the user enters the Work Type ID and clicks Fetch Story

Backend Changes

- Create Jira authentication logic
- Validate connection using API credentials
- Fetch user stories from Jira
- Pass fetched data into the existing workflow for generating test cases

AI-Assisted Development Workflow

- Use GitHub Copilot inside Visual Studio Code
- Provide prompts for codebase analysis
- Identify impacted files and required changes
- Implement step by step with approval-based iterations

3. Tools Used

- Visual Studio Code
- GitHub Copilot
- Atlassian Jira
- Existing project codebase
- LLM-assisted prompt-based development workflow

4. Understanding Gained

During this assignment, I learned the following:

AI-Assisted Development (Vibe Coding)

- Learned how AI coding assistants can analyze an existing codebase
- Understood how to write effective prompts for analyzing and implementation tasks in VS
- Learned iterative development by asking Copilot to proceed step by step with approval
- Understood how AI can accelerate development while maintaining developer control

Prompt Engineering

- Learned how prompt quality directly affects code generation
- Understood how to provide clear implementation instructions to AI coding assistants
- Learned how structured prompts help identify impacted files and architecture changes

Existing Codebase Analysis

- Learned how to understand an unfamiliar project structure
- Understood how frontend and backend components interact in an application
- Learned how to identify files requiring modification before implementation

API Integration

- Learned how Jira API authentication works using email and API key
- Understood request-response flow while connecting external APIs
- Learned how external systems can be integrated into applications

LLM Application Development

- Understood how enterprise tools like Jira can be integrated into AI-powered workflows

- Learned how retrieved Jira user stories can be used as structured input for generating outputs such as test cases

5. Implementation Process Followed

1. Opened the project in Visual Studio Code
2. Used GitHub Copilot to analyze the existing codebase
3. Prompted Copilot to identify impacted files and architecture changes
4. Reviewed suggested changes before implementation
5. Added frontend UI for Jira connection
6. Implemented Connect Jira functionality
7. Created backend logic for Jira authentication
8. Established secure Jira API connection
9. Retrieved user stories from Jira
10. Populated user stories into the application UI
11. Connected fetched stories to the existing test case generation workflow
12. Tested end-to-end integration with iterative AI-assisted development

6. Challenges Faced

- Understanding the structure of an existing codebase
- Writing effective prompts for GitHub Copilot
- Reviewing AI-generated suggestions before implementation
- Mapping Jira data into the existing application workflow
- Ensuring integration did not break existing features

7. Outcome

Successfully integrated Jira into the existing application using AI-assisted development workflow.

The application can now:

- Connect to Jira using API credentials

The image displays two screenshots of a web application titled "User Story to Tests" running on localhost:5173. The application's subtitle is "Generate comprehensive test cases from your user stories".

The first screenshot shows the "Jira Connection" section. It includes a "Jira Base URL" field with the value "https://anithapraji.atlassian.net/", an "Email" field with the value "anitha.praji@gmail.com", and an "API Token" field with masked characters. A "Show" link is visible next to the API Token field. A "Connect to Jira" button is at the bottom right. A status indicator in the top right corner shows a red dot and the text "Disconnected".

The second screenshot shows the same interface after a successful connection. The "Jira Connection" section now displays the "Base URL: https://anithapraji.atlassian.net/", "Email: anitha.praji@gmail.com", and "Session ID: jira_1782386115494_n...". A "Disconnect" button is now present. Below this section, a new "Fetch Story from Jira" section is visible, featuring an "Issue ID or Key" field with the placeholder text "e.g., PROJ-123 or 12345" and a "Fetch Story" button. The status indicator in the top right corner now shows a green dot and the text "Connected".

- Retrieve user stories automatically

localhost:5173

Fetch Story from Jira

Issue ID or Key

SRSTRSSCRM-12

Fetch Story Clear

SRSTRSSCRM-12 Story

Nurse Dashboard View Request

DESCRIPTION

As a user, I want to nurse dashboard view request so that the related functionality works as expected.

✓ ACCEPTANCE CRITERIA

Given a user with valid access, when they open the dashboard, then relevant data and visualizations should be displayed correctly without delay.

Acceptance Criteria from Jira

Given a user with valid access, when they open the dashboard, then relevant data and visualizations should be displayed correctly without delay.

- Display stories in the UI

localhost:5173

✓ Story Details (from Jira) Jira Imported

Story Title *

Nurse Dashboard View Request

Description

As a user, I want to nurse dashboard view request so that the related functionality works as expected.

Acceptance Criteria * ✓ (from Jira)

Given a user with valid access, when they open the dashboard, then relevant data and visualizations should be displayed correctly without delay.

Additional Info

Source: Jira Story (SRSTRSSCRM-12)

Generate

- Use Jira data for test case generation workflow

localhost:5173

Generated Test Cases			
(30 test cases) generated • Model Name: LLaMA-3.1-70B-Venice • Input Tokens: 474 • Output Tokens: 2147 • Cost: \$0.002718 (est.)			
Test Case ID	Title	Category	Expected Result
TC-001	Valid User Access	Positive	Relevant data and visualizations are displayed correctly.
TC-002	Invalid User Access	Negative	Access is denied and an error message is displayed.
TC-003	No Data Available	Edge	A no data available message is displayed.
TC-004	Large Data Set	Edge	Data and visualizations are displayed correctly without delay.
TC-005	User Authentication	Authorization	User has the required role and permissions to access the nurse dashboard.
TC-006	Page Load Time	Non-Functional	Page load time is within the acceptable limit (less than 5 seconds).
TC-007	Data Refresh	Non-Functional	Data is refreshed at the specified interval (e.g., every 5 minutes).
TC-008	Browser Compatibility	Non-Functional	The nurse dashboard is displayed correctly across different browsers.
TC-009	Device Compatibility	Non-Functional	The nurse dashboard is displayed correctly across different devices.
TC-010	Network Connectivity	Non-Functional	The nurse dashboard is displayed correctly with varying network connectivity.
TC-011	User Session Timeout	Non-Functional	The user is logged out after the session timeout interval.
TC-012	Multiple User Sessions	Non-Functional	The nurse dashboard is displayed correctly for each user session.
TC-013	Error Handling	Non-Functional	The error is handled and displayed correctly to the user.
TC-014	Security Vulnerabilities	Non-Functional	The nurse dashboard is secure and does not contain vulnerabilities.
TC-015	Audit Logging	Non-Functional	Audit logs are generated and contain the correct information.
TC-016	Data Encryption	Non-Functional	Data is encrypted in transit and at rest, and encryption protocols are up-to-date and secure.
TC-017	Backup and Recovery	Non-Functional	Backups are performed regularly, and data can be recovered in case of a failure.
TC-018	Compliance with Regulations	Non-Functional	The nurse dashboard complies with relevant regulations, and data is handled and stored in accordance with regulations.
TC-019	User Feedback	Non-Functional	User feedback is collected and used to improve the nurse dashboard.
TC-020	Performance Metrics	Non-Functional	Performance metrics are collected and used to improve the nurse dashboard.
TC-021	Accessibility	Non-Functional	The nurse dashboard is accessible for users with disabilities, and accessibility features are implemented.
TC-022	Localization	Non-Functional	The nurse dashboard is localized for different languages and regions, and localization features are implemented.
TC-023	Customization	Non-Functional	The nurse dashboard can be customized for different user roles and preferences, and customization features are implemented.
TC-024	Integration with Other Systems	Non-Functional	The nurse dashboard integrates with other systems, and integration features are implemented.
TC-025	Scalability	Non-Functional	The nurse dashboard can handle a large number of users and data, and scalability features are implemented.
TC-026	Disaster Recovery	Non-Functional	Disaster recovery plans are in place, and data can be recovered in case of a disaster.
TC-027	Business Continuity	Non-Functional	Business continuity plans are in place, and the nurse dashboard can continue to operate in case of an outage.
TC-028	Vendor Support	Non-Functional	Vendor support is available for the nurse dashboard, and vendor support is responsive and effective.
TC-029	Training and Documentation	Non-Functional	Training and documentation are available for the nurse dashboard, and training and documentation are effective and up-to-date.
TC-030	Continuous Improvement	Non-Functional	Continuous improvement is in place for the nurse dashboard, and improvement is effective and identifies issues promptly.

- Support faster implementation using AI coding assistance

8. Key Learnings

From this assignment, I learned:

- AI-assisted development using GitHub Copilot
- Prompt engineering for coding tasks
- Working with existing codebases
- API integration with Jira
- Frontend and backend communication
- Secure authentication handling
- Step-by-step implementation workflow using AI assistance
- Building LLM-powered applications integrated with enterprise systems

9. Conclusion

This assignment helped me understand how modern software development can be accelerated using AI-assisted coding tools such as GitHub Copilot.

It improved my understanding of prompt engineering, codebase analysis, API integration, iterative development, and how enterprise systems like Jira can be connected with LLM-powered applications for practical real-world use cases.